

Unit test project: Drop chip game

Drop chip game

Instruction

The drop chip game, connect 4, is a two-player connection game in which the two players 'X' and 'O' take turns dropping chips from the top into a seven-column, six-row vertically suspended grid. The pieces fall straight down, occupying the lowest available space within the column. The objective of the game is to be the first to form a horizontal, vertical, or diagonal line of four of one's own chips.

See **main.py** for the implemented Python class for the Connect 4 game. The two methods in the **main.py** file are:

1. **drop_chip(column: int) -> bool**: This method should take a column number (1-7) as an argument and drop a chip of the current player's chip into the specified column. The chip should occupy the lowest available space within the column. If the column is full or the column number is out of range, the method should return False. Otherwise, it should switch the current player and return True.
2. **check_win(player: str) -> bool**: This method should take a player ('X' or 'O') as an argument and return True if the player has four of their chips in a row horizontally, vertically, or diagonally on the board. Otherwise, it should return False.

Project requirements

Languages/Technologies:

- Python
- unittest

Team Size: This is an individual project.

Project Timeline:

- two weeks.
- Project part 1: complete and submit by Sunday, October 13, before midnight.
- Project part 2: complete and submit by Sunday, October 20, before midnight.

Grading:

- Project part 1: 15 points
- Project part 2: 15 points

Project description:

Part 1: Analyzing the Connect4 Class in main.py

Learning outcome: The objective of this project is to thoroughly analyze the Connect4 class defined in the main.py file. This process will involve examining the functionality of each method within the class, running them independently, and understanding how each behaves in isolation. The analysis will include modifying class values, observing the impact of these changes, and documenting findings.

Steps to Complete the Analysis:

1. **Access the main.py file:** Open and review the contents of the file to familiarize yourself with the class structure.
2. **Download the file unit_test_project_part1.docx:** This document will be used to record your observations, changes made to the class, and results obtained from running the class.
3. **Analyze each class:**
 - Run the class independently.
 - For each method, understand how it works by testing its functionality.
 - Make modifications to some values in the class and observe how these changes affect the behavior of the class and the game logic.
4. **Document the analysis:** In the Word document (unit_test_project_part1.docx):
 - Record how each method behaves when run.
 - Note the modifications made (e.g., changing player symbols, adjusting the board size, etc.).
 - Describe the outcome of these changes and any other observations regarding the class's behavior.
5. **Conclude the analysis:** Summarize the key takeaways from the modifications and their effects on the game's mechanics.

Part 2: creating a unit test

After completing **Part 1**, where the functionality of the Connect4 class was analyzed, **Part 2** involves creating comprehensive unit tests to ensure the correctness of key game functions. These tests will be designed to verify the game's win conditions and chip-dropping logic.

1. **Create Unit Tests for Win Conditions:**
 - Develop unit tests to check the various win conditions in the Connect4 game.

- **Test cases** should include:
 - **Horizontal win:** Ensure the game correctly detects a horizontal line of four matching chips.
 - **Vertical win:** Test that a vertical line of four matching chips results in a win.
 - **Diagonal win:** Verify that diagonal lines, both top-left to bottom-right and bottom-left to top-right, are correctly identified as wins.
 - **No win:** Ensure that the game correctly continues if there is no winning combination after a move.

2. Create Unit Tests for Chip Dropping Logic:

- Write unit tests to verify the behavior of the `drop_chip` method.
- **Test cases** should include:
 - **Successful chip drop:** Ensure that a chip is successfully dropped into an available column.
 - **Full column:** Test that the method returns `False` when trying to drop a chip into a column that is already full.
 - **Invalid column:** Ensure the method handles invalid columns (i.e., column numbers out of range) properly.
 - **Full board:** Test how the game behaves when the board is full, and no more chips can be dropped.

Steps to Complete Part 2:

1. **Set up a unit testing framework:** Use Python's `unittest`.
2. **Write unit tests:** Implement test cases to cover both the win conditions and chip-dropping logic as described above.
3. **Run the tests:** Execute the unit tests and verify that all tests pass successfully. If any tests fail, investigate the reason and document the issue.
4. **Document the results:** Record your test results, including any bugs or issues identified during the testing process, in the project documentation. The documentation should be at the end Python file as comments.